

Technical Briefing on Phoenix Security's Graph Technologies, Data Models, and genAI Integration

Executive briefing and scope

Phoenix Security positions its platform as an “Actionable ASPM” (Application Security Posture Management) capability that centralises vulnerability data from multiple scanners, enriches it with threat intelligence, and drives prioritised remediation actions across “code-to-cloud” environments. ¹ The core differentiator repeatedly emphasised across Phoenix Security's public materials is the use of relationship-centric representations—described as a “Cyber Risk Graph”, “risk and attack-based graph”, and (more recently) a “Vulnerability Intelligence Graph”—to connect findings to ownership, runtime context, business impact, and threat intelligence. ²

This briefing focuses on (a) the publicly evidenced technology surface area (documentation, public repos, and product communications), (b) Phoenix Security's graph-based data model and the graph operations implied by its feature set, and (c) how the graph layer aligns with and enables Phoenix's stated agentic/genAI roadmap (including MCP and IDE integrations). Where Phoenix does not publicly disclose implementation specifics (e.g., the underlying database engine), this briefing explicitly distinguishes evidence from inference and frames open validation questions suitable for an internal architecture review.

Core products and services through a graph lens

Phoenix Security's externally described product surface clusters into a few graph-reliant capabilities: correlation across domains, ownership attribution, attack-path/blast-radius reasoning, and intelligence-driven prioritisation.

Phoenix markets a “Cyber Risk Graph of your Software Assets” and highlights a “risk and attack-based graph” intended to connect vulnerabilities to applications and deployments so teams can “see the vulnerabilities deployed and where to ACT”. ³ This is reinforced by pricing/packageing that explicitly lists “Risk Graph” and “Application Composition with Risk graph”. ⁴

A second major capability area is ownership and organisational mapping: Phoenix describes correlating vulnerabilities to their “rightful owner” and mapping business units, teams, and applications affected by a vulnerability (example: Log4j) via Blast Radius Analysis. ⁵ The explicit value proposition is that graph-driven correlation reduces ambiguity (“Who owns this risk, and how bad is it?”) by traversing relationships from a vulnerability/finding to impacted organisational units. ⁵

A third capability area is code-to-cloud contextualisation and deduplication. Phoenix states it aggregates and deduplicates vulnerabilities “from code to cloud”, and separately documents “contextual deduplication” (including canary token-based traceability) as a mechanism to reduce noise and treat duplicates differently depending on deployment context. ⁶

Finally, Phoenix's roadmap increasingly exposes a combined "graph + intelligence + agentic actions" approach. Phoenix 3.30 introduces a "Vulnerability Intelligence Graph" that links vulnerabilities to campaigns, libraries, business units, teams, applications, assets, and individual findings, and then merges that organisational mapping with threat layers such as exploitability, threat actors, and attack techniques. ⁷ This is the clearest public statement that Phoenix is moving from "graph as visualisation" to "graph as an intelligence substrate" spanning both enterprise context and external threat knowledge. ⁷

Publicly evidenced architecture and technology stack components

External interfaces, integrations, and ingestion surface

Phoenix Security's knowledge base and marketing content describe a connector-driven ingestion model: Phoenix "connects to your security scanners to retrieve vulnerability data", enriches it with threat intelligence, and provides a "single asset register across Cloud, Container, software, infrastructure and applications." ⁸ The integrations catalogue explicitly frames Phoenix as integrating into "native technology stacks" across AppSec, cloud, containers, and collaboration tools. ⁹

From the open-source side, the **Phoenix Security PYRUS** repository provides an unusually concrete view of Phoenix's configuration and ingestion automation approach. PYRUS is described as a YAML-native "CMDB automation framework" that connects metadata from repositories, CI/CD pipelines, cloud environments, and identity systems to maintain a continuously updated view of applications, containers, and services. ¹⁰ The same repo documents an API-driven workflow that creates teams, applications, components (repos), cloud environments, and rules—suggesting the platform exposes a relatively rich API model for asset and ownership configuration. ¹¹

Notably, PYRUS documentation identifies a Phoenix API endpoint pattern (`https://api.YOURDOMAIN.securityphoenix.cloud`) and lists validated `AssetType` values spanning software-facing components and infrastructure-facing services (e.g., `REPOSITORY`, `FOSS`, `SAST`, `CONTAINER`, `CLOUD`). ¹¹ This explicit taxonomy strongly implies that Phoenix's internal model is multi-domain and relationship-heavy by design, regardless of the backing persistence layer.

Correlation, attribution, and "living CMDB" automation

Phoenix's PYRUS launch materials describe a "YAML-native engine" that reads metadata from repos, clouds, identity providers, and portals; auto-groups assets by service and owner; and "feeds Phoenix Security's ASPM to route the right vulnerability to the right team— with self-healing configuration when people, code, or environments change." ¹² The PYRUS blog post further anchors integrations with ServiceNow, Backstage, Okta, GitHub/GitLab/Bitbucket (CODEOWNERS), and cloud tags/Kubernetes metadata for automated grouping and attribution. ¹³

From an architecture standpoint, these statements imply at least four distinct pipeline stages:

- multi-source metadata ingestion (identity + SCM + cloud + CMDB/service catalog),
- normalisation/entity-resolution into canonical "service/app/component/environment/team" entities,
- rule-based grouping and ownership propagation ("self-healing configuration"),
- downstream enrichment of findings with ownership + business context used for routing and dashboards. ¹⁴

Runtime context: reachability, lineage, and blast radius

Phoenix describes “reachability analysis” as both static and runtime, used to decide whether vulnerable components are actually executed/used, and positions this as a means to reduce false positives and prioritise what matters. ¹⁵ Separately, Phoenix announces “container lineage and throttling” as an “agentless code 2 cloud container lineage” capability intended to connect build artefacts to runtime containers and reduce alert fatigue by excluding inactive images/containers. ¹⁶

Blast Radius Analysis then appears as the organisational analogue of lineage: mapping vulnerability impact across business units, teams, and applications, and enabling drill-down from vulnerability → impacted org structures. ⁵

Architecture component map with evidence vs open questions

Layer / component	Primary responsibilities	Public evidence	Notes for internal validation
Ingestion & integration connectors	Pull/import findings from scanners; ingest metadata from SCM/CI/CD/cloud/IdP/CMDB	Integrations catalogue; FAQ on scanner connectivity; PYRUS integration claims ¹⁷	Which ingress protocols dominate (push vs pull)? Which schemas are canonical internally (finding vs vulnerability vs asset)?
Asset & ownership modelling	Canonical applications, components, services, environments, teams; ownership propagation	PYRUS repo + launch posts; supported <code>AssetType</code> taxonomy ¹⁸	Is the canonical model stored as a graph store, relational schema, or “dual model” (relational + graph projection)?
Correlation & contextual deduplication	Merge duplicates across SDLC and deployment contexts; adjust risk based on correlation density	Phoenix 3.30 notes; reachability/dedup claims; “canary token-based traceability” ¹⁹	What is the dedup key strategy (SBOM coordinates, file hashes, container lineage IDs, scanner-specific IDs)?
Graph reasoning layer	Traversals/queries for blast radius, attack paths, vulnerability intelligence linking	Blast Radius post; “risk and attack-based graph”; Vulnerability Intelligence Graph release ²⁰	What query language/engine is used for graph operations? Are path queries computed on-demand or pre-materialised?
Risk scoring engine	Multi-tier risk aggregation across vulnerabilities → assets → services/components → business units	Phoenix risk formula article; threat-centric + 4D framing ²¹	How are time-series risk metrics stored and recomputed when graph edges change (e.g., ownership drift)?

Layer / component	Primary responsibilities	Public evidence	Notes for internal validation
Presentation & workflow integration	Dashboards, vulnerability explorer, exports, ticketing, collaboration	Dashboard upgrade notes; team dashboards; workflow tool integration (Jira/Azure DevOps/Slack) 22	How much of “graph” is interactive UI vs API-accessible primitives?

Graph technologies and data models implemented by Phoenix Security

What Phoenix means by “graph” in public materials

Phoenix Security uses “graph” in at least three distinct—but related—ways:

- A **risk/asset graph**: “Cyber Risk Graph of your Software Assets” and “risk and attack-based graph” for code-to-cloud visibility and prioritisation. 3
- An **organisational impact graph**: Blast Radius Analysis mapping vulnerabilities to business units, teams, and applications. 5
- An **intelligence graph**: the “Vulnerability Intelligence Graph” linking vulnerabilities to campaigns, libraries, org structures, assets, and individual findings, plus threat intelligence overlays (threat actors, exploitability, attack techniques). 7

Crucially, Phoenix does not publicly name a specific graph database product (e.g., Neo4j, Neptune). The safest conclusion from public sources is therefore: Phoenix implements **graph-based data models and graph-driven product features**, but the **underlying storage/query technology remains undisclosed**.

Canonical entities implied by PYRUS and platform releases

Across PYRUS artefacts and Phoenix platform announcements, the platform’s canonical “nodes” and “edges” can be reconstructed with high confidence, because Phoenix explicitly names the objects being created, grouped, and linked.

Key node families:

- **Software construction**: Repository/Component, FOSS/SBOM elements, Build artefacts, SAST results. 11
- **Runtime & infrastructure**: Container, Cloud resources/services, Environments. 23
- **Security artefacts**: Vulnerability, Finding (and correlation groups/deduplicates), Campaigns. 7
- **Org model**: Teams, Business units, Owners/maintainers (via IdP and repo ownership). 24
- **Threat intelligence**: CVE/CWE, threat actors, attack techniques, exploitability evidence. 25

Key edge families (representative):

- APPLICATION contains COMPONENT (repo); components relate to vulnerabilities/findings. 26

- **COMPONENT/BUILD** produces **CONTAINER_IMAGE**; **CONTAINER_IMAGE** runs_in **ENVIRONMENT** (lineage). ²⁷
- **FINDING** maps_to **VULNERABILITY**; **VULNERABILITY** linked_to **CVE** / **CWE** and threat intel. ²⁸
- **VULNERABILITY** impacts **APPLICATION** / **BUSINESS_UNIT** / **TEAM** (blast radius). ²⁹
- **VULNERABILITY** associated_with **CAMPAIGN** and **ATTACK_TECHNIQUE** / **THREAT_ACTOR** (intelligence graph). ²⁵

Diagram: inferred Phoenix “risk + vulnerability intelligence” graph model

```

flowchart LR
    subgraph Org[Organisation & ownership]
        BU[Business Unit]
        T[Team]
        Owner[Owner/Maintainer]
    end

    subgraph SDLC[Code & build]
        App[Application]
        Repo[Component / Repository]
        Lib[Library / FOSS package]
        Build[Build Artifact]
    end

    subgraph Runtime[Runtime & infra]
        Img[Container Image]
        Env[Environment]
        Cloud[Cloud Service/Resource]
    end

    subgraph Sec[Security objects]
        Finding[Finding]
        Vuln[Vulnerability]
        CVE[CVE]
        CWE[CWE]
        Camp[Campaign]
    end

    subgraph TI[Threat intelligence]
        Actor[Threat Actor]
        TTP[Attack Technique / TTP]
        Expl[Exploitability Evidence]
    end

    BU --> T
    T --> Owner

    BU --> App
    App --> Repo
    Repo --> Lib
    Repo --> Build
  
```

```

Build --> Img
Img --> Env
Env --> Cloud

Finding --> Vuln
Vuln --> CVE
Vuln --> CWE

Vuln --> Camp
Vuln --> Expl
Expl --> Actor
Expl --> TTP

```

This diagram is not a claim about Phoenix's internal database schema; it is a consolidation of relationships Phoenix explicitly describes: organisational mapping and "blast radius" across business units/teams/apps, code-to-cloud lineage, and the Vulnerability Intelligence Graph layer connecting vulnerabilities to campaigns, CVEs/CWEs, and threat intelligence. ³⁰

What graph operations Phoenix's features require

Phoenix's product claims imply specific graph computations that senior engineers/architects can use as a checklist when evaluating current implementation trade-offs.

Phoenix feature	Public description	Graph operation implied	Why graph-native modelling matters
Cyber Risk Graph / risk & attack-based graph	"single risk based graph" and "risk and attack-based graph" for code-to-cloud visibility ³¹	Multi-hop traversals from findings → assets → apps → deployments; subgraph queries by scope	Relationships change frequently (deployments, ownership); graph model reduces impedance for traversal-heavy queries
Blast Radius Analysis	Maps "which business units, teams, and applications are affected" by a vulnerability ⁵	Reverse reachability ("what depends on/contains/uses this?"); impact propagation and aggregation	Requires traversing outward from a vulnerability node across ownership + composition edges
Container lineage & throttling	"agentless code 2 cloud container lineage... detect which build file gets built and run where" ¹⁶	Provenance/path queries (repo → build → image → runtime); filtering inactive nodes	Lineage is naturally a directed graph; provenance queries are awkward in flat asset tables

Phoenix feature	Public description	Graph operation implied	Why graph-native modelling matters
Vulnerability Intelligence Graph	Links vulns to “campaigns, libraries, business units, teams, applications, assets... CVEs, CWEs, and threat intelligence... threat actors, attack techniques” 7	Knowledge-graph style joins; neighbourhood exploration; cross-domain correlation	A single connected model enables correlation that would otherwise require many brittle joins across domain tables
Contextual deduplication	Dedup and risk adjustment “based on correlation and deployment type” 32	Entity resolution + cluster formation; edge-weighted scoring by linkage counts	Dedup groups are subgraphs; the “shape” of linkage becomes a risk factor
PYRUS ownership automation	Auto-group assets by owner/service; “self-healing configuration” 12	Graph rewrite / edge reattachment when ownership nodes change	Ownership drift becomes a graph maintenance problem; “self-healing” suggests automated refactoring of link structure

How Phoenix’s graph usage integrates with genAI workflows

Phoenix’s stated AI agent architecture

Phoenix publicly describes an agentic model with three cooperating agents: **Researcher**, **Analyzer**, and **Remediator**. The Researcher monitors threat intelligence and maps attack methodologies (explicitly referencing MITRE ATT&CK), correlating vulnerabilities to ransomware campaigns and threat actors; the Analyzer “models attack paths within the actual code-to-cloud environment”; and the Remediator turns this combined intelligence into environment-specific actions and remediation plans. 33

The more detailed “AI-agent-second” narrative makes a critical architectural point: Phoenix claims agents run **after** attribution/context is built (“context is the compass”), describing “4-dimensional risk attribution” (ownership, deployment location, exposure level, business context) and then applying role-specific agents once data is “related, contextualized, deduplicated, and attributed.” 34

This ordering is highly aligned with graph-first design: the agent prompt/context window can be assembled from a bounded, relevant subgraph (the vulnerable component, its lineage, its owning team, its runtime exposure, and relevant threat intel), rather than forcing an LLM to reason over unstructured lists of findings.

MCP, IDEs, and “vibe coding” integrations

Phoenix explicitly states an intention to integrate remediation into modern IDE and “vibe coding” workflows via an upcoming **MCP server**, naming Cursor, Lovable, and GitHub Copilot as targets for connecting vulnerability remedy workflows. 35

From an architecture perspective, the “graph + MCP” combination implies an emerging interaction pattern:

1. Developer/agent requests context for a vulnerability (or a set of vulnerabilities affecting a repo/service).
2. Phoenix returns a graph-derived context bundle: affected repos/components, reachable runtime paths, deployment environments, ownership, SLA context, and threat intel mappings.
3. The IDE agent proposes code/config changes, and Phoenix workflows then track remediation state and impact on risk metrics.

Phoenix’s own content reinforces the “graph-context reduces hallucination/noise” argument: it explicitly criticises dumping large vulnerability sets into LLMs and describes the Analyzer as mapping the attack path, correlating threat modelling, and using reachability and container lineage signals to narrow to what is actually reachable and active. ³⁴

genAI integration points tied to graph primitives

genAI workflow stage	Phoenix capability (public)	Graph primitive feeding the LLM/agent	Expected output artefacts
Threat intel triage	Researcher agent monitoring CTI and mapping vulnerabilities to threat actors/techniques ³³	Subgraph: <u>Vuln ↔ Exploitability ↔ Actor/TTP ↔ Campaign</u>	Prioritised watch lists, exploitability narrative, campaign tagging
Environment-specific risk reasoning	Analyzer modelling attack paths and chaining potential; reachability + lineage awareness ³³	Subgraph: <u>Repo/Lib → Build → Image → Env → Exposure</u> plus related vulnerabilities	“Is it reachable?” decision; blast radius; inferred exploit paths
Remediation planning	Remediator producing remediation plans and grouped actions ³⁶	Vulnerability clusters (connected components) plus ownership/priority properties	Bundled work items (“plan of record”), compensating controls, PR-ready instructions
Workflow execution	Ticketing/workflow integration (Jira/Azure DevOps/ServiceNow referenced in materials) ³⁷	Edges from findings/vulns to owners and remediation artefacts	Tickets with accurate routing + scoped remediation context
IDE/MCP integration	Stated MCP/IDE roadmap ³⁵	API-exposed graph query endpoints packaged as MCP tools	In-editor contextual guidance, autofix suggestions, validation loops

Novelty relative to common “LLM + vuln list” approaches

The novel element in Phoenix’s publicly stated approach is not merely “LLM summarisation”; it is the claim that Phoenix’s agents depend on a pre-existing contextual substrate that includes ownership

attribution, deduplication/correlation, reachability, container lineage, and threat-intel mappings—most naturally represented and queried as a graph. ³⁸ The explicit MCP integration path also suggests Phoenix intends to operationalise this context inside developer tools, rather than keeping AI confined to a dashboard. ³⁹

Comparison to common graph tools and frameworks

What Phoenix’s use-cases demand from “graph technology”

From Phoenix’s described features, the graph system (regardless of implementation) must support:

- High-churn relationship updates (deployments, repos, ownership drift), consistent with PYRUS “self-healing” and metadata-driven updates. ¹²
- Traversal-heavy queries (blast radius, lineage, attack paths). ⁴⁰
- Cross-domain joins between internal asset context and external threat knowledge (Vulnerability Intelligence Graph). ⁷
- Time-series aggregation layered on top of the graph (risk evolution charts, exposure over time), implying either a dual-store design or time-aware graph properties. ⁴¹

Industry comparison table: implementation strategies Phoenix could align with

Because Phoenix’s underlying storage is not publicly disclosed, the comparison below is framed as **architectural options for implementing Phoenix’s described graph features**, rather than an attribution of Phoenix’s current internal choices.

Graph implementation approach	Typical technologies (examples)	Strengths for Phoenix-described needs	Trade-offs / risks
Native property graph database	Graph DB engines optimised for traversals and subgraph queries	Natural fit for blast radius, lineage/provenance, neighbourhood exploration (Vulnerability Intelligence Graph), and evolving schema of node/edge types	Operational maturity, multi-tenant isolation, and analytics/time-series often require additional components
RDF / knowledge graph stack	Triple stores + ontology-driven modelling	Good fit when semantics/ontologies are first-class (e.g., representing threat intel and its relationships with formal meaning)	Can be heavier-weight for high-churn operational asset graphs; bridging to path-centric “attack graphs” may require careful modelling choices
Relational core + graph projection	Tables for entities/edges; graph computed in memory or via specialised services	Strong for reporting, compliance exports, and transactional updates; can co-exist with existing SaaS app data models	Traversal queries can become expensive/complex; risk of duplicating logic in multiple query layers

Graph implementation approach	Typical technologies (examples)	Strengths for Phoenix-described needs	Trade-offs / risks
Search/analytics engine + relationship overlays	Index-driven analytics plus relationship metadata	Strong for full-text threat intel and dashboard analytics; can accelerate “find all occurrences” queries	Harder to deliver deep multi-hop traversals and path logic without a dedicated graph layer

Where Phoenix’s public direction appears strongest

Independent of the underlying database choice, Phoenix’s public releases suggest a deliberate convergence toward a unified, multi-domain relationship model:

- PYRUS focuses on **automated ownership and grouping**, effectively ensuring the org/asset portion of the graph stays correct despite change. ¹²
- Container lineage focuses on **build-to-runtime provenance**, filling a common blind spot in many vulnerability platforms where “repo findings” and “runtime findings” remain weakly linked. ¹⁶
- The Vulnerability Intelligence Graph explicitly adds **campaign/threat actor/TTP overlays**, creating the substrate needed for high-quality agentic actions and “context-first” AI. ²⁸

Targeted questions for an internal engineering/architecture review

To turn this external analysis into a senior-architect-level internal briefing discussion, the following questions are the most leverage-heavy (and directly implied by Phoenix’s own public claims):

- **Graph persistence and query engine:** What is the authoritative storage for the connected model (graph-native store vs relational + projection), and what query interfaces (internal DSL/API) power blast radius, lineage, and intelligence graph features?
- **Schema governance:** How are node/edge types versioned as Phoenix expands from “risk graph” to “vulnerability intelligence graph” while maintaining backwards compatibility for dashboards, exports, and MCP tools? ⁴²
- **Deduplication correctness:** How does contextual deduplication interact with lineage (repo→image→runtime) so that “one vulnerability” remains correctly represented across multiple deployment contexts without losing the ability to compute blast radius and ownership routing? ⁴³
- **Agent context packaging:** What is the contract for generating “bounded, minimal, correct” context bundles for agents (Researcher/Analyzer/Remediator and MCP/IDE clients), and how is sensitive data (e.g., asset exposure, internal architectures) redacted or scoped? ³⁵

These questions align directly with Phoenix’s stated trajectory: making graph-backed context operationally consumable by both humans (dashboards/graphs) and agents (MCP + remediation automation). ⁴⁴

¹ ⁶ ³¹ Phoenix Security (former Phoenix Security) Contextual Application Security and Cloud Security Vulnerability Management (Main Landing) - Phoenix Security
<https://phoenix.security/phoenix-security-aspm/>

- 2 3 **Phoenix Security - What is ASPM? how it differ from ASOC**
<https://phoenix.security/phoenix-security-aspm-what-it-is-and-what-does-it-cover/>
- 4 **Pricing**
https://phoenix.security/pricing/?utm_source=chatgpt.com
- 5 20 24 29 30 40 **Phoenix Security ASPM Feature Focus: Blast Radius and Business Unit Impact Analysis Documentation - Phoenix Security**
<https://phoenix.security/phoenix-security-blast-radius/>
- 7 19 25 28 32 42 43 44 **ASPM Application Security Vulnerability Management – Phoenix 3.30 Release**
<https://phoenix.security/aspm-application-security-vulnerability-management-phoenix-3-30/>
- 8 **Phoenix Security FAQ**
https://kb.phoenix.security/?ht_kb=faq&utm_source=chatgpt.com
- 9 17 **Integration**
https://phoenix.security/integrations/?utm_source=chatgpt.com
- 10 11 23 26 **GitHub - Security-Phoenix-demo/Phoenix-Security-PYRUS: PYRUS is Phoenix Security's YAML-native CMDB automation framework that unifies asset ownership, vulnerability attribution, and business alignment across DevSecOps. It replaces static CMDBs with metadata-driven synchronization from CI/CD, repositories, cloud, and identity systems to reflect the real state of your environment**
<https://github.com/Security-Phoenix-demo/Phoenix-Security-PYRUS>
- 12 18 **ASPM Application Security | Phoenix Security Launches PYRUS for Code-to-Cloud Remediation**
<https://phoenix.security/pyrus-yaml-aspm-application-security-vulnerability-management/>
- 13 14 **ASPM Vulnerability Management Reinvented with Phoenix PYRUS**
<https://phoenix.security/aspm-application-security-vulnerability-management-pyrus-yaml-automation/>
- 15 **The Ultimate Guide to Reachability Analysis Which reachability is good for you: Enhancing Code, Library, and Container Security with ASPM - Phoenix Security**
<https://phoenix.security/aspm-reachability-analysis-overview/>
- 16 27 **The Missing Link in Container Security: Lineage, Contextual Deduplication, and Throttling in Phoenix Security ASPM - Phoenix Security**
<https://phoenix.security/feat-container-lineage/>
- 21 **Phoenix Security Risk Formula details**
https://phoenix.security/phoenix-security-act-on-risk-calculation/?utm_source=chatgpt.com
- 22 41 **Phoenix Security Unveils ASPM Dashboard Upgrade: Switch Between Code & Cloud, Customize Risk Views, and Quantify Exposure - Phoenix Security**
<https://phoenix.security/phoenix-security-aspm-new-design-to-switch-code-cloud-and-bring-customization/>
- 33 36 **Phoenix AI Agents Redefine ASPM and Vulnerability Management**
<https://phoenix.security/phoenix-ai-agents-aspm-vulnerability-management/>
- 34 38 **AI-Agent: A Human-Centric Use of AI Agents to Fix ASPM and Vulnerability Management - Phoenix Security**
<https://phoenix.security/ai-second-agents/>
- 35 39 **Phoenix AI Agents for ASPM & Vulnerability Managemen**
<https://phoenix.security/phoenix-ai-agents-aspm-vulnerability-management-2/>
- 37 **Phoenix Security - SMART Vulnerability Managment**
<https://phoenix.security/application-security/>